Car Rental Booking System

ReactJS + NestJS + (RDBMS/NoSQL) + Swagger/OpenAPI

We need to build a Zoomcar/Enterprise-style car rental booking application. Users search cars by pickup location, date range and car type, add rental add-ons (insurance, GPS, child seat) and book with mock payment. Car owners list their cars and view bookings. Focus on date-range availability, add-on pricing and per-day cost calculation.

Area	Requirement
Frontend	ReactJS, React Router, React Query, Material UI
Backend	NestJS, REST APIs, DTO validation, JWT authentication
Database	ANY (RDBMS / NoSQL)
API Documentation	Swagger / OpenAPI — Mandatory
Architecture	Controller → Service → Repository/Model → DB
Suggested Duration	3-4 Days
Version Control	GitHub

1. Project Objective

Build a production-style car rental platform where users book a car for a date range with add-ons, availability is server-enforced (no double booking on overlapping dates), and owners manage inventory and view bookings.

Learning Objectives

- React date-range pickers and dynamic add-on selection.
- Filter UI (transmission, fuel, seats, price).
- NestJS modules, controllers, services and DTO validation.
- JWT authentication and role-based authorization.
- Overlapping date-range availability queries.
- Multi-part price computation (per day + add-ons + tax).
- Swagger / OpenAPI documentation.

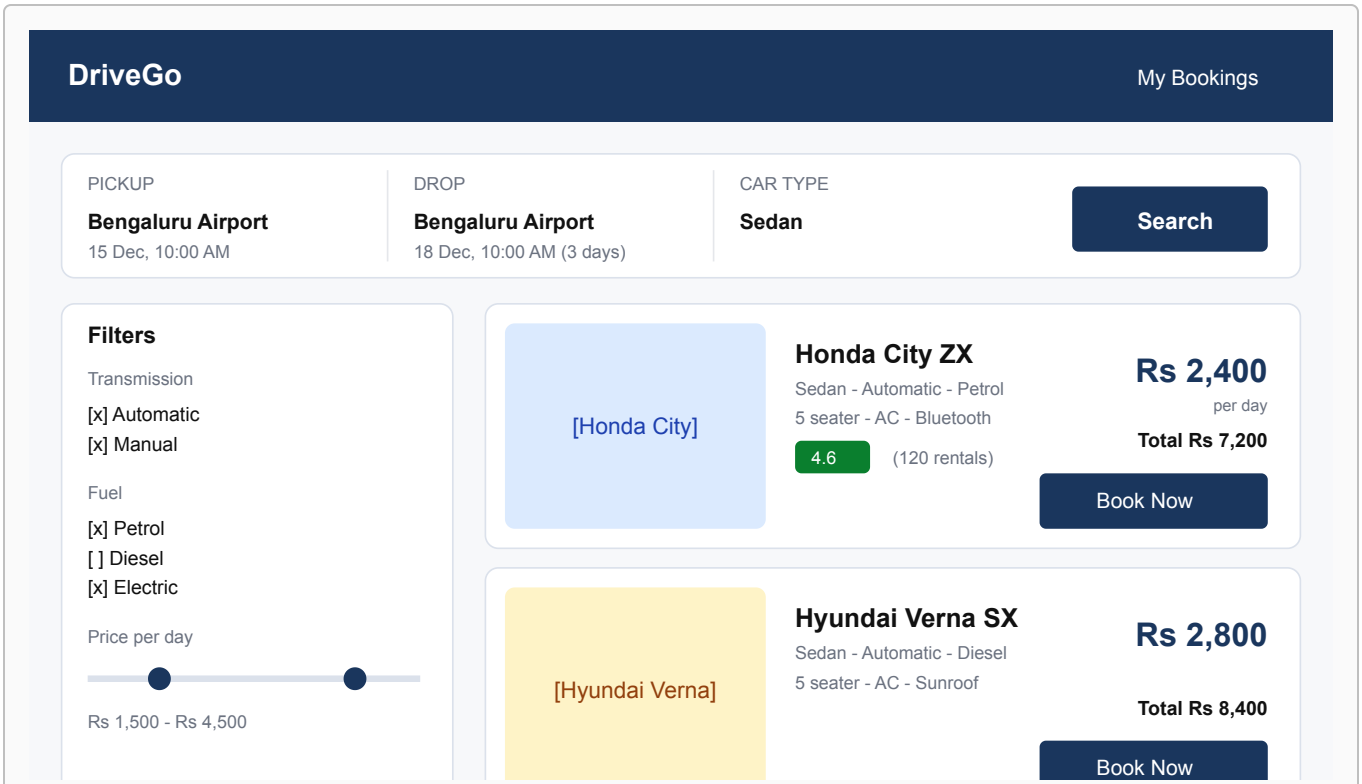
2. User Roles

Role	Responsibilities

Renter	Search and book cars, add add-ons, view / cancel own bookings, write review.
Car Owner	List own cars with pricing and locations, view bookings for own cars.
Admin	Approve car listings and manage users; view all bookings.

3. Visual Reference

Use the embedded mockup as visual and information-architecture reference. Styling may be improved while preserving the required functionality.



Search Results — filters on the left, cars with per-day price and total for the selected date range.

4. Core Workflow

When a renter submits a rental booking, the backend performs:

1. Validate request, dates and add-on selection.
2. Ensure pickup datetime is in the future and drop-off is after pickup.
3. Query for any existing booking on the same car overlapping the requested range.
4. If overlap found, reject with 409 CONFLICT.
5. Compute number of rental days server-side.
6. Compute total: (per-day price × days) + sum(add-on prices × days) + tax.
7. Persist the booking with status CONFIRMED and reference number.
8. Create notifications for owner and renter and an activity log entry.

5. Business Rules

- Pickup must be later than the current server time.
- Drop-off must be after pickup and within the maximum allowed rental duration.
- Same car cannot be booked by two renters for overlapping periods.
- Total price is always recomputed on the server.
- Cancellation is allowed up to a configurable window before pickup (e.g. 24 hours).
- A car must be APPROVED by admin to appear in search.
- Owners cannot rent their own cars.
- Deleting a car is not allowed if it has upcoming bookings.

6. Data Model

The data model may be built in a relational database or a NoSQL store (choice of the developer). The main entities that must exist:

```
User          { id, name, email, phone, passwordHash, drivingLicense, role }
Car           { id, ownerId, make, model, year, type, transmission, fuel, seats, ci
AddOn         { id, name, pricePerDay }
Booking       { id, carId, renterId, pickupAt, dropOffAt, days, subtotal, addOnsTota
BookingAddOn  { id, bookingId, addOnId, pricePerDaySnapshot }
Review        { id, bookingId, carId, renterId, rating, comment, createdAt }
```

7. REST API Requirements

API endpoints must be meaningful, RESTful and role-protected. At minimum the following capabilities are required:

- Auth: signup, login, refresh, get current user.
- Cars: search (city + dates + filters), get, owner CRUD, admin approve.
- Add-ons: list, admin CRUD.
- Bookings: create, list own, get, cancel.
- Owner: list bookings for own cars.
- Reviews: create for completed booking, list for a car.

8. Swagger / OpenAPI — Mandatory

Every API endpoint must be documented in Swagger. The Swagger UI should be exposed at a route such as `/api/docs`.

- JWT Bearer authentication.

- Request DTO schemas with required/optional fields.
- Response schemas.
- HTTP status codes and error responses.
- Endpoint descriptions and examples.
- Query parameters for pagination, filtering, sorting and search.
- Role/authorization requirements where applicable.

9. ReactJS Screens

Screen	Requirements
Login / Signup	JWT login with role-based redirect.
Home / Search	City, pickup date, drop-off date, car type.
Search Results	Car cards + filter sidebar (transmission, fuel, price).
Car Detail	Photos, specs, per-day price, add-ons picker, total preview.
Booking Review	Selected add-ons, price breakdown, driver details form.
Booking Confirmation	Success screen with ref number and pickup instructions.
My Bookings	Upcoming and past, cancel with confirmation.
Owner: My Cars	List / add / edit cars with images.
Owner: Bookings	Bookings for owner's cars, upcoming and past.

10. Search, Filtering and Pagination

The main list API should support: `page`, `limit`, `search`, `city`, `pickupAt`, `dropOffAt`, `type`, `transmission`, `fuel`, `seats`, `minPrice`, `maxPrice`, `sortBy`, `sortOrder`.

11. Reporting / Dashboard

- Bookings per car (utilisation) per month.
- Revenue per car and per owner.
- Most-booked car type / brand.
- Cancellation rate.
- Average rating per car.

12. Advanced Logic — Optional

- Coupon / promo codes.
- Driver-included option with hourly rate.

- Delivery to home address (with fee).
- Damage report upload on drop-off.
- Loyalty points on completed rentals.
- Email / SMS reminders 24 hours before pickup.

13. Definition of Done

- All required screens are functional.
- APIs are protected according to user roles.
- Swagger documentation is complete and usable.
- Indexes are added for important query fields.
- Pagination/filtering works server-side.
- Business rules are implemented in the service layer, not controllers.
- Activity/audit history is maintained where applicable.
- Core business logic is covered by tests.
- README contains setup instructions, environment variables and Swagger URL.

Final Expected Outcome

You should be able to build and explain a full-stack car rental app with correct date-range availability, add-on pricing, no-double-booking guarantee, owner workflow and complete Swagger documentation.